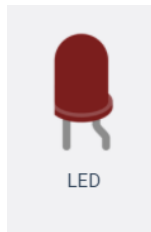


Dimmable LED Selector Project

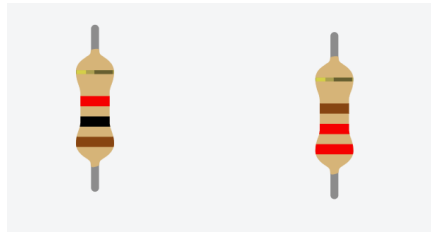
We've had several class projects that work with **light** and **basic user** input. Now it's your turn to see what you can do on your own!

Complete this project initially in a simulated environment. Once you have that working successfully, put together your physical circuit.

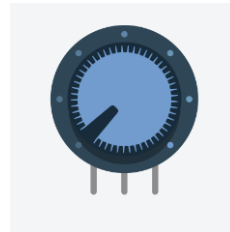
Materials



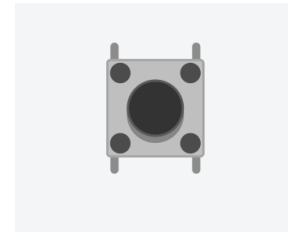
4 x LEDs
Any Colors



Resistors:
4 x 220 Ω or 330 Ω



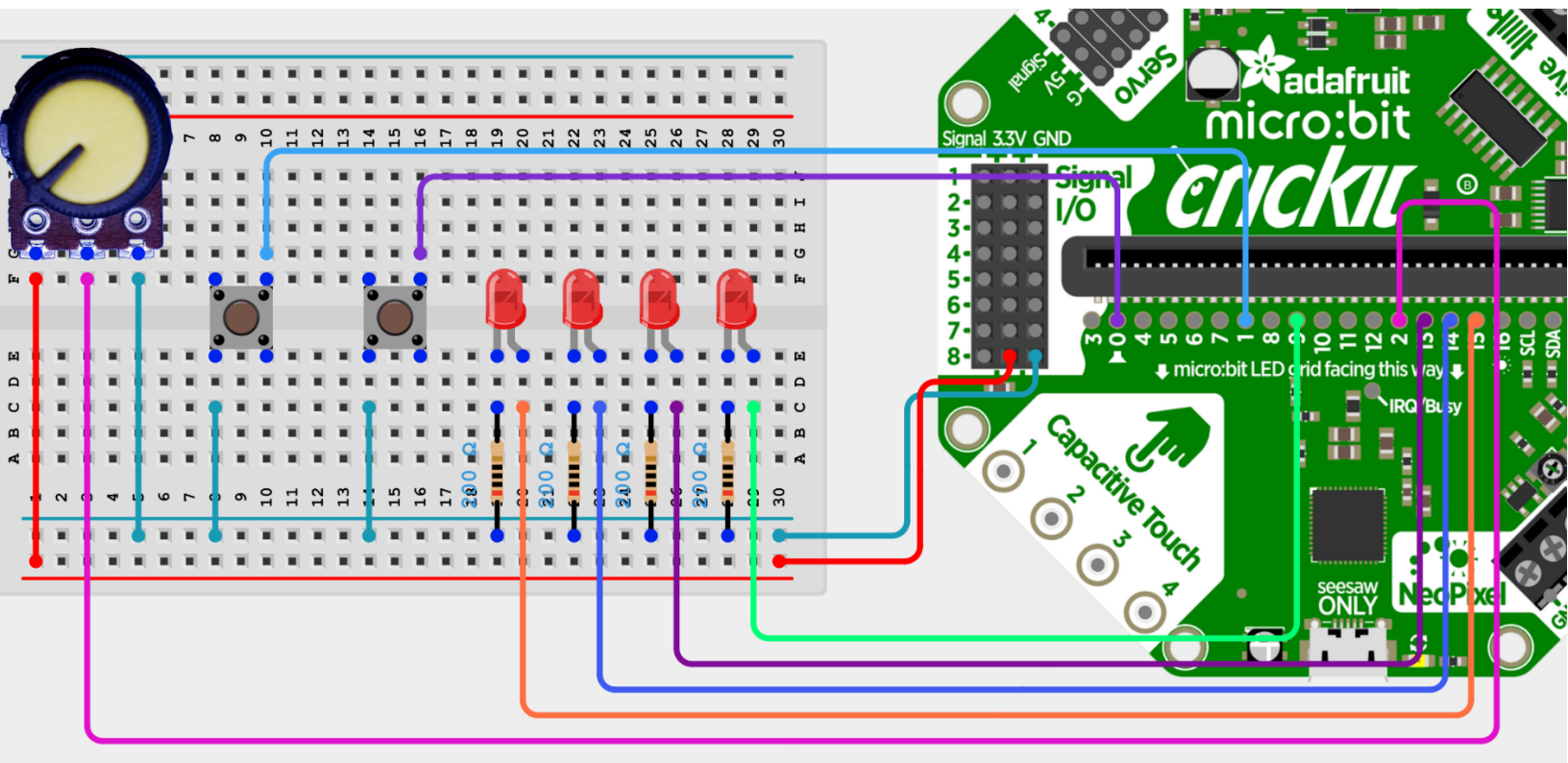
1 x Variable Resistor
(potentiometer)



2 x Pushbutton

Wiring Diagram

Wire your project according to the following diagram.



Verify Circuit

As the complexity of this circuit is increasing, so is the possibility of errors in component placement (or poor connections for the physical circuit). Before getting into the complexity of the whole project, it would be a good idea to **verify that you've wired the components correctly**, by individually running a simple test on each.

Note: *As you test each of the three sections, you should **save your code** so that you can also easily run the same tests when you get to making the physical circuit.*

Test One: LEDS

You should have four LEDs wired into pins 9, 12, 13, and 14. To test them, just try turning each pin to a **HIGH Voltage State** using either `digitalWrite()` or `analogWrite()`.



Test Two: Buttons

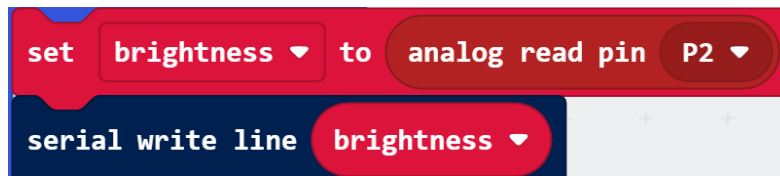
We have two buttons to test, which should be connected to pins 0 and 1. These will be used to change which LED is on; one button will move the LED **one position to the left** and the other will move it **to the right**.

In order to test you've wired the buttons correctly, use the **Serial Monitor** to print a simple message when each button press is detected:



Test Three: Variable Resistor

You have a single variable resistor with which we can use an **analogRead()** to gain some insight about its current angular position. Store the position in a variable and print it out. This should yield values between 0-1023 when rotating the potentiometer.



Later on, we'll want our values to lie between 0-255, so make the next adjustment to **map()** the values to this new number range:



----- ensure all three tests are successful before moving on -----

Assignment Details: Base Features

For this assignment, completing the following base features will earn you a maximum mark of 9/10. Some hints and explanation will be given, but you'll need to construct a code solution on your own.

It may be very useful to look back at your previous demos as reference material.

One LED on:

Although you have four LEDs, in this circuit we only want **one to be illuminated at a time**. The user will be able to (with button presses) switch which button is on.

To build in this functionality, you'll likely want to create a variable (perhaps called **currentLED**) to remember which LED should be on.

- Since there are 4 LEDs, you should ensure this variable is always one of 4 possible values:

One Possible Scheme:

- 0 → bottom LED should be on
 - 1 → second LED from bottom should be on
 - 2 → third LED from bottom should be on
 - 3 → fourth LED from bottom should be on
- Remember that when one LED is on, all others should be off!

Switch which LED is on:

Using the two pushbuttons (momentary switches), allow the user to change which LED is active. Follow the pattern shown in the **wiring schematic** where:

- the top button will move the active LED up
- the bottom button will move the active LED down

If the code that turns the LEDs on and off relies on a single variable (**currentLED**), you should only need to modify that variable by adding or subtracting from it when a button is pressed.

- Remember to deal with wrap-around values
 - If currentLED goes below zero, wrap around to 3
 - If currentLED goes above 3, wrap around to 0

Dim the active LED with Potentiometer:

Using your potentiometer, allow the brightness of the active LED to be modified from **off** up to **full brightness**.

Recall that we can change the intensity of an LED using the **analogWrite** block (rather than digitalWrite, which can only do on/off)

Assignment Details: Expert Challenge

For the remaining 1/10 grade, modify your program to implement one of the following:

Two LED on:

Modify your code so that a group of 2 LEDs will always be on and can be still moved with the pushbuttons.

- The two illuminated LEDs should always be side-by-side, **except** when wrapping around

Variable Active LEDs:

Change the behavior of your potentiometer to instead control **how many LEDs can be active at once**.

Use the following scheme:

- First 25% (0 – 63 mapped values): One Active LED
- Second 25% (64 – 127 mapped values): Two Active LEDs
- Third 25% (128 – 191 mapped values): Three Active LEDs
- Fourth 25% (192 – 255 mapped values): Four Active LEDs